

Survival-Trajectory Phenotypes from Individualized Survival Curves with `unsurv`

Validity projection, time-weighted integral distances, and medoid clustering for predicted survival curves.

Imad EL BADISY

2026-08-24

Table of contents

Input representation	1
Validity projection	2
Time-weighted integral distances	2
Medoid clustering	3
Prediction for new patients	4
Stability	4
Example	4
Plots	7
A structural limitation worth carrying forward	9
Interpretation	9

Unsupervised phenotyping usually clusters molecular or clinical covariates and tests survival differences post hoc, which optimizes a covariate representation rather than the time-to-event prediction itself, and can miss structure expressed mainly through when risk accumulates. Many survival learners instead output individualized survival trajectories $\hat{S}_i(t)$ rather than only a scalar risk score. `unsurv` clusters those trajectories directly: the input is a survival probability matrix, not the covariate matrix.¹

Input representation

Let n be the number of individuals and $t_1 < \dots < t_Q$ a strictly increasing time grid. For individual i , the observed or predicted curve is $\mathbf{S}_i = (S_i(t_1), \dots, S_i(t_Q))$, stacked into a matrix $S \in [0, 1]^{n \times Q}$. A mathematically valid survival curve lives in

¹`unsurv` package documentation, CRAN: <https://CRAN.R-project.org/package=unsurv>.

$$\mathcal{S}_Q = \{\mathbf{u} \in [0, 1]^Q : u_1 \geq u_2 \geq \dots \geq u_Q\},$$

bounded and non-increasing. The goal is to partition the n trajectories into K clusters by similarity of temporal survival behavior, working directly with S rather than with the covariates that produced it.

Validity projection

Predicted curves from a flexible learner are not guaranteed to land in $\mathcal{S}_Q$: small local irregularities can push values outside $[0, 1]$ or briefly break monotonicity. `unsurv` applies a deterministic map $\Pi : \mathbb{R}^Q \rightarrow \mathcal{S}_Q$ before computing any distance.

Clamping bounds each value to $[0, 1]$,

$$S_i(t_q) \leftarrow \min\{1, \max\{0, S_i(t_q)\}\}.$$

Monotonicity enforcement then sweeps left to right, pulling each point down to no more than the already-corrected previous point,

$$S_i(t_q) \leftarrow \min(S_i(t_q), S_i(t_{q-1})), \quad q = 2, \dots, Q,$$

which guarantees the non-increasing property by construction rather than by post hoc rejection. An optional median smoothing step with odd window width $w \geq 3$ can follow,

$$S_i(t_q) \leftarrow \text{median}\{S_i(t_r) : r \in [q - h, q + h]\}, \quad h = (w - 1)/2,$$

useful when curves come from a learner flexible enough to show small local wiggles that carry no real signal.

Time-weighted integral distances

Distances are built to approximate normalized continuous-time integrals over the follow-up window, not raw pointwise sums. With $\Delta_q = t_{q+1} - t_q$, trapezoidal quadrature weights are

$$w_1 = \frac{1}{2}\Delta_1, \quad w_q = \frac{1}{2}(\Delta_{q-1} + \Delta_q) \quad (2 \leq q \leq Q - 1), \quad w_Q = \frac{1}{2}\Delta_{Q-1},$$

normalized so that $\sum_q w_q = 1$. For an L_2 distance, curves are mapped into a weighted Euclidean space by $x_{iq} = \sqrt{w_q} S_i(t_q)$, so that ordinary squared Euclidean distance recovers the discretized integral,

$$\|\mathbf{x}_i - \mathbf{x}_j\|_2^2 = \sum_{q=1}^Q w_q (S_i(t_q) - S_j(t_q))^2 \approx \frac{1}{t_Q - t_1} \int_{t_1}^{t_Q} (S_i(t) - S_j(t))^2 dt.$$

Taking the square root of the quadrature weight before differencing is what lets a plain Euclidean norm on the transformed matrix stand in for a time integral of a squared term. For an L_1 distance the weight is applied without the square root, since the absolute-value term is linear rather than quadratic, $x_{iq} = w_q S_i(t_q)$, giving

$$\|\mathbf{x}_i - \mathbf{x}_j\|_1 = \sum_{q=1}^Q w_q |S_i(t_q) - S_j(t_q)| \approx \frac{1}{t_Q - t_1} \int_{t_1}^{t_Q} |S_i(t) - S_j(t)| dt.$$

Optional per-column standardization can follow the weighting step when early and late follow-up regions sit on very different scales, as can happen with long follow-up.

Medoid clustering

Given the embedded feature matrix, **unsurv** computes a pairwise dissimilarity matrix, Euclidean for L_2 and Manhattan for L_1 , and applies partitioning around medoids.² For medoid indices $M = \{m_1, \dots, m_K\}$, the fitted partition minimizes

$$\sum_{i=1}^n \min_{m \in M} d(\mathbf{x}_i, \mathbf{x}_m).$$

Each cluster prototype under this scheme is an observed curve, not an averaged one, so the reported prototype is always a real, valid patient trajectory, and the method accepts any dissimilarity rather than requiring one compatible with a centroid update. If K is not specified, **unsurv** scans a candidate range and selects the value maximizing mean silhouette width, a pragmatic exploratory default rather than an inferential test for the number of clusters.

```
Require: S, time grid, preprocessing options, distance type, K or K_max
Apply clamping; optionally enforce monotonicity and smoothing
Compute trapezoidal weights
Construct the weighted feature matrix under L1 or L2 mapping
Compute pairwise dissimilarities
if K unspecified: select K by silhouette scanning
Fit PAM; return labels and medoids
```

²Kaufman, L., and Rousseeuw, P. J. (1990). *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.

Prediction for new patients

For a new curve $\mathbf{S}_{\text{new}}$ on the same time grid, **unsurv** applies the same validity map and feature embedding Φ used at training time, then assigns the nearest medoid,

$$\hat{c}(\mathbf{S}_{\text{new}}) = \arg \min_k d(\Phi(\Pi(\mathbf{S}_{\text{new}})), \Phi(\Pi(\mathbf{S}_{m_k}))).$$

This is well defined precisely because medoids are stored observed curves from training rather than a fitted functional form, so a new patient is assigned without refitting anything.

Stability

Because predicted curves inherit uncertainty from the upstream survival learner, **unsurv** reports stability under resampling, subsampling without replacement or bootstrap with replacement, with optional perturbations such as additive noise on curves followed by monotonicity enforcement, perturbation of the time weights, or feature-space jitter. Stability is summarized by the Adjusted Rand Index between resampled partitions on overlapping individuals, comparing the resampled clustering against the reference partition.

Example

This is the synthetic data-generating process used for the main figure of the **unsurv** application note: three latent risk regimes are simulated, **survdnn** learns individualized survival curves from covariates without ever seeing the latent regime label, and **unsurv** recovers the regimes from those predicted curves alone.³

Ninety patients per regime are drawn from cluster-specific Weibull event-time distributions with progressively larger shape and scale parameters across regimes, plus four covariates whose distributions also shift by regime and which perturb each patient’s individual Weibull parameters, so the latent structure stays recoverable but is not a deterministic function of the observed covariates alone.

```
library(survival)
library(survdnn)
library(unsurv)
library(ggplot2)
library(patchwork)

set.seed(123)
```

³EL BADISY, I. (2026). *unsurv: Clustering Individualized Survival Curves*. Application note, Bioinformatics Advances. Synthetic example and code reproduced from the manuscript’s supplementary appendix.

```

simulate_weibull_time <- function(shape, scale) {
  scale * (-log(stats::runif(length(shape))))^(1 / shape)
}

n_per_cluster <- 90
cluster_specs <- data.frame(
  true_cluster = 1:3,
  shape = c(1.08, 1.35, 1.70),
  scale = c(6.0, 11.5, 19.5),
  x1_mean = c(-2.4, 0.0, 2.4),
  x2_mean = c(-1.6, 0.8, 2.8),
  x3_prob = c(0.20, 0.50, 0.80),
  x4_mean = c(2.0, 0.3, -1.2)
)

sim_list <- vector("list", nrow(cluster_specs))
for (i in seq_len(nrow(cluster_specs))) {
  spec <- cluster_specs[i, ]
  x1 <- rnorm(n_per_cluster, spec$x1_mean, 0.55)
  x2 <- rnorm(n_per_cluster, spec$x2_mean, 0.60)
  x3 <- rbinom(n_per_cluster, 1, spec$x3_prob)
  x4 <- rnorm(n_per_cluster, spec$x4_mean, 0.50)

  shape_i <- pmax(0.8, rnorm(n_per_cluster, spec$shape + 0.03 * x3 - 0.02 * x4, 0.07))
  scale_linpred <- 0.20 * x1 + 0.18 * x2 + 0.35 * x3 - 0.12 * x4
  scale_i <- spec$scale * exp(0.10 * scale_linpred + rnorm(n_per_cluster, 0, 0.06))

  event_time <- simulate_weibull_time(shape_i, scale_i)
  censor_time <- runif(n_per_cluster, 10, 28)
  sim_list[[i]] <- data.frame(
    true_cluster = spec$true_cluster,
    time = pmin(event_time, censor_time),
    status = as.integer(event_time <= censor_time),
    x1 = x1, x2 = x2, x3 = x3, x4 = x4
  )
}
dat <- do.call(rbind, sim_list)

```

A `survdnn` model is fit on a 70 percent training split using only the observed (`time`, `status`) outcome and the four covariates, never the latent regime label, and used to predict survival curves for the held-out 30 percent on a common 100-point grid up to the 90th percentile of training follow-up.

```

idx_train <- sample.int(nrow(dat), floor(0.70 * nrow(dat)))
train <- dat[idx_train, ]
test <- dat[-idx_train, ]

mod <- survdnn(
  Surv(time, status) ~ x1 + x2 + x3 + x4,
  data = train,
  hidden = c(24, 12), activation = "relu",
  dropout = 0, batch_norm = FALSE,
  epochs = 160, lr = 5e-4,
  loss = "aft", optimizer = "adamw",
  optim_args = list(weight_decay = 1e-5),
  .seed = 123, verbose = FALSE
)

t_max <- as.numeric(quantile(train$time, 0.90, na.rm = TRUE))
time_grid <- seq(0.1, t_max, length.out = 100)
S_test <- as.matrix(predict(mod, newdata = test, type = "survival", times = time_grid))

```

unsurv then clusters the predicted test-set curves into $K = 3$ groups, with monotonicity enforcement and light median smoothing against small local irregularities in the network output.

```

fit <- unsurv(
  S = S_test, times = time_grid, K = 3, K_max = 3,
  distance = "L2", enforce_monotone = TRUE, smooth_median_width = 3,
  standardize_cols = FALSE, eps_jitter = 0, seed = 123
)
fit

```

```

unsurv (PAM) fit
  K:3
 distance:L2 silhouette_mean:0.732
  n:81 Q:100

```

Cluster labels from PAM are arbitrary integers, not ordered by risk, so they are relabeled by each medoid's row mean, a simple average-survival-level summary, before comparing against the known latent regime. Recovery is scored with the Adjusted Rand Index, computed directly from the contingency table rather than via `unsurv_compare()` (which compares partitions using observed outcomes, not against a known simulation label).

```

comb2 <- function(x) ifelse(x < 2, 0, x * (x - 1) / 2)
adjusted_rand_index <- function(labels_true, labels_pred) {
  tab <- table(labels_true, labels_pred)
  n <- sum(tab)
  sum_ij <- sum(comb2(tab)); sum_i <- sum(comb2(rowSums(tab))); sum_j <- sum(comb2(colSums(tab)))
  expected <- sum_i * sum_j / comb2(n)
  max_index <- 0.5 * (sum_i + sum_j)
  if (max_index == expected) return(1)
  (sum_ij - expected) / (max_index - expected)
}

cluster_order <- order(rowMeans(fit$medoids))
relabel <- setNames(seq_along(cluster_order), cluster_order)
pred_cluster <- unname(relabel[as.character(fit$clusters)])

contingency <- table(True = factor(test$true_cluster, levels = 1:3),
                     Predicted = factor(pred_cluster, levels = 1:3))
contingency

```

```

      Predicted
True  1  2  3
  1 26  0  0
  2  0 26  1
  3  0  0 28

```

```

accuracy <- sum(diag(contingency)) / sum(contingency)
ari <- adjusted_rand_index(test$true_cluster, pred_cluster)
c(accuracy = round(accuracy, 3), ari = round(ari, 3))

```

```

accuracy    ari
    0.988    0.962

```

Recovery is close to exact: a single class-2 patient is placed in class 3, for a label-matched accuracy of 0.988 and an Adjusted Rand Index of 0.962, computed entirely from curves predicted on held-out patients, with the true regime label never used by either `survdnn` or `unsurv`.

Plots

The left panel is the raw model output passed to `unsurv()`, before clustering; the right panel is the same curves colored by the recovered phenotype with medoid prototypes overlaid, produced by `unsurv`'s own `plot()/autoplot()` method on the fitted object.

```

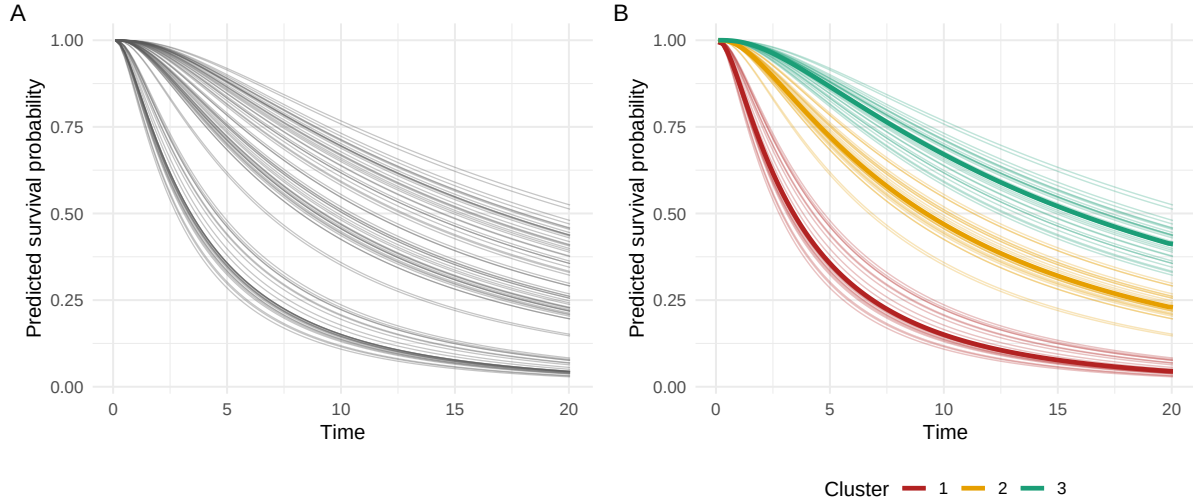
curve_df <- data.frame(
  patient_id = rep(seq_len(nrow(S_test)), each = length(time_grid)),
  time = rep(time_grid, times = nrow(S_test)),
  surv = as.vector(t(S_test)),
  cluster = factor(rep(pred_cluster, each = length(time_grid)), levels = 1:3)
)
S_med <- fit$medoids[cluster_order, , drop = FALSE]
med_df <- data.frame(
  time = rep(time_grid, times = nrow(S_med)),
  surv = as.vector(t(S_med)),
  cluster = factor(rep(seq_len(nrow(S_med)), each = length(time_grid)), levels = 1:3)
)

fig_left <- ggplot(curve_df, aes(time, surv, group = patient_id)) +
  geom_line(linewidth = 0.35, alpha = 0.35, color = "grey35") +
  theme_minimal(base_size = 12) +
  labs(x = "Time", y = "Predicted survival probability") +
  theme(legend.position = "none")

fig_right <- ggplot(curve_df, aes(time, surv, group = patient_id, color = cluster)) +
  geom_line(linewidth = 0.40, alpha = 0.28) +
  geom_line(data = med_df, aes(time, surv, group = cluster, color = cluster), linewidth = 1.0) +
  theme_minimal(base_size = 12) +
  scale_color_manual(values = c("1" = "#B22222", "2" = "#E69F00", "3" = "#1B9E77")) +
  labs(x = "Time", y = "Predicted survival probability", color = "Cluster") +
  theme(legend.position = "bottom")

fig_left + fig_right + plot_annotation(tag_levels = "A")

```

Panel A alone gives no indication of temporal-risk subgroups; panel B recovers three visually and numerically distinct trajectory groups directly from the predicted curves, without using the observed test-set outcomes at any point, illustrating the intended **unsurv** workflow: an upstream survival learner maps covariates to individualized trajectories, and **unsurv** converts those trajectories into interpretable temporal phenotypes.

A structural limitation worth carrying forward

Under a proportional-hazards model, every predicted curve is $S_i(t) = S_0(t)^{\exp(\eta_i)}$, a scalar power transform of one shared baseline. Any L_1 or L_2 curve-shape distance between two such curves reduces to a monotone function of the scalar linear predictor η_i alone, so clustering curve shapes under proportional hazards is mathematically equivalent to clustering a single number. **unsurv** is doing genuinely curve-native work only for learners that do not impose proportional hazards, such as random survival forests or a **survdnn** model fit with the AFT or Cox-Time loss, where curve shape carries information beyond one scalar summary.

Interpretation

A practical reporting template is:

- Cluster 1: slow decline in predicted survival, high expected event-free time.
- Cluster 2: intermediate decline, moderate expected event-free time.
- Cluster 3: rapid decline, low expected event-free time.

After clustering, each phenotype should be described with survival probabilities at clinically meaningful horizons, restricted mean survival time summaries, and baseline covariate distributions, so the phenotype stays anchored to survival behavior while remaining interpretable to

clinical collaborators. Partitions can be compared against alternative clusterings, such as one built on a scalar risk summary or on covariate principal components, with `unsurv_compare()`, which reports cluster sizes, Adjusted Rand Index agreement, and per-cluster Kaplan-Meier medians rather than a log-rank test, since testing a partition against the labels it was fit to separate is circular.